Table 1 — ESP32 I²C stress test results (measured from serial output)

(Each row = Freq / Msg size / inter-packet gap → Throughput (B/s) | Msg/s | Error (%))

Freq (Hz)	Size (B)	Gap (ms)	Throughput (B/s)	Msg/s	Error (%)
100000	10	0	7916.7	158.33	5.00
100000	10	10	905.0	90.50	0.0
100000	50	0	7851.2	157.02	5.0
100000	50	10	3333.3	66.67	0.0
400000	10	0	0.00	0.00	100.0
400000	10	10	0.00	0.00	100.0
400000	50	0	0.00	0.00	100.0
400000	50	10	0.00	0.00	100.0

Table 2 — Recommended configuration (shortlist with brief rationale)

Freq (Hz)	Size (B)	Gap (ms)	Rationale (brief)
100000	50	10	Sweet spot: reasonable throughput (3333.3 B/s), low message rate (66.67 msg/s) and 0% error. Good when you need medium payloads reliably.
100000	10	10	Lowest-error option for small messages: 90.5 msg/s at 0% error, but lower total bytes/s (905 B/s). Use if small frequent messages are required.
100000	50	0	Highest throughput measured (7916.7 B/s, 158.3 msg/s) but shows 5% error — use only if you can tolerate occasional packet loss and want maximum throughput.
400000	any	any	Not recommended (observed failures / 100% error). Only try after hardware/wiring/pull-up/firmware changes.

Recommended single configuration (overall “sweet spot”):

I²C 100 kHz, 50 B messages, 10 ms gap. This balances payload throughput and reliability (0% error). If your application prioritises throughput above all and can accept some loss, 100 kHz, 50 B, 0 ms gap gives the highest bytes/sec but with a measurable error rate.

Short discussion — choosing & justifying the sweet spot

1. **Key trade-offs visible in the data**
    - At **100 kHz** you successfully delivered payloads with measurable throughput. The 50 B, gap=0 run gives the highest throughput but shows **~5% error**, indicating the link or device cannot maintain that continuous load without occasional failures.
    - Adding a **10 ms gap** drastically reduced or eliminated errors: 50 B, gap=10 at 100 kHz gave **0% errors** while throughput dropped to a third of the gap=0 case. That shows the bus/slave needs small idle time to process/acknowledge or to recover buffer/interrupt handling.
    - At **400 kHz** the device produced **no valid messages** in your capture (sent=0 valid=0), so the high-speed mode appears unsupported, misconfigured, or the wiring/termination/pull-ups/firmware timing are insufficient.
2. **Why 100 kHz / 50 B / 10 ms is the “sweet spot”**
    - **Reliability first:** many embedded control apps require 100% of critical messages to arrive. The 50 B / 10 ms result shows **0% error**, so it’s safe.
    - **Reasonable throughput:** 3.3 kB/s is acceptable for moderate telemetry or command batches, while still allowing the slave MCU time to process each packet.
    - **Simplicity:** Standard-mode (100 kHz) tends to be the most robust across different devices and wiring conditions.
3. **Why not 400 kHz (now)?**
    - Your capture shows **no successful transactions** at 400 kHz. Common causes: inadequate pull-ups (higher speed requires stronger pull-ups), long wire length, poor signal integrity, slave not supporting 400 kHz, or timing/ACK issues in the slave firmware. Before using 400 kHz you must fix hardware/firmware issues and re-test.
4. **Practical suggestions to improve performance (and to possibly enable 400 kHz):**

- Ensure proper **pull-up resistors** on SDA/SCL (typical 2.2k–4.7k depending on bus capacitance and voltage). For 400 kHz you often need lower resistor values than for 100 kHz.
- Keep wiring short and avoid noisy traces; add proper ground connection between master and slave.
- Verify the **slave device actually supports 400 kHz** and that your slave firmware acknowledges/handles quick consecutive packets (interrupts, buffer sizes).
- Add small **gap (ms)** between packets if slave processing is slow — your experiments show this reduces error.
- Add **retransmit / ACK checks** and an application-layer CRC so you can recover from brief errors.
- If you need higher reliable throughput than 100 kHz allows, consider an alternative physical layer (SPI, UART at high baud, or hardware-assisted DMA transfers) rather than forcing 400 kHz I^2C.